Code Optimization

1

Content

- Need (rationale) for Optimization
- Problems facing optimization
- Classification of Optimization
  - Scope e.g. Peephole, basic block, loop, global, whole-program
  - Language (in)dependency
  - Machine (in)dependency
- Factors influencing optimization e.g. Machine architecture, cpu architecture etc.
- Optimization transformations

2

IR Optimization

Goal: Improve the IR generated by the previous step to take better advantage of resources.

- One of the most important and complex parts of any modern compiler.
- A very active area of research.

3

Sources of Optimization

- In order to optimize our IR, we need to understand why it can be improved in the first place.
- Reason one: IR generation introduces redundancy.
  - A naïve translation of high-level language features into IR often introduces subcomputations.
  - Those subcomputations can often be sped up, shared, or eliminated.
- Reason two: Programmers are lazy.
  - Code executed inside of a loop can often be factored out of the loop. e.g

```
while (x < y + z) {
  x = x – y;
}
```

  - Language features with side effects often used for purposes other than those side effects.

4

The challenge of optimization

- A good optimizer
  - Should never change the observable behavior of a program.
  - Should produce IR that is as efficient as possible.
  - Should not take too long to process inputs.
- Unfortunately:
  - Even good optimizers sometimes introduce bugs into code.
  - Optimizers often miss “easy” optimizations due to limitations of their algorithms.
  - Almost all interesting optimizations are NP-hard or undecidable.

5

What are we optimizing?

- Optimizers can try to improve code usage with respect to many observable properties.
- What are some quantities we might want to optimize?
  - Runtime (make the program as fast as possible)
  - Memory usage (generate the smallest possible executable)
  - Power consumption (choose simple instructions at the expense of speed and memory usage)
- Note for these quantities, there are trade-offs of time, memory usage, power etc.
- Plus a lot more (minimize function calls, reduce use of floating-point hardware, etc.)

6

IR vs. Code optimization

- There is not always a clear distinction between what belongs to "IR optimization" versus "code optimization."
- Typically:
  - IR optimizations try to perform simplifications that are valid across all machines.
  - Code optimizations try to improve performance based on the specifics of the machine.

7

Code Optimization

REQUIREMENTS:

- Meaning must be preserved (correctness)
- Speedup must occur on average.
- Work done must be worth the effort.

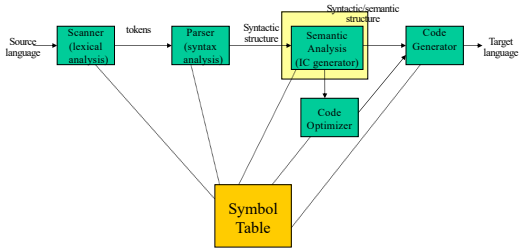
OPPORTUNITIES:

- Programmer (algorithm, directives)
- Intermediate code
- Target code

8

8

Code Optimization



9

9

Levels

- Window – peephole optimization
- Basic block – local
  - An optimization is local if it works on just a single basic block.
- Procedural – global (control flow graph)
  - An optimization is global if it works on an entire control-flow graph.
- Program level – intraprocedural (program dependence graph)
  - An optimization is intraprocedural if it works across the control-flow graphs of multiple functions

10

10

Peephole Optimizations

- Local in nature
- Pattern driven
- Limited by the size of the window

11

11

Peephole Optimizations

- Constant Folding

  `x := 32` becomes `x := 64`

  `x := x + 32`
- Unreachable Code

  `goto L2`

  `x := x + 1` ← unneeded
- Flow of control optimizations

  `goto L1` becomes `goto L2`

  …

  `L1: goto L2`

12

12

Peephole Optimizations

- Algebraic Simplification
  `x := x + 0` ← unneeded
- Dead code
  `x := 32` ← where x not used after statement
  `y := x + y` → `y := y + 32`
- Reduction in strength
  `x := x * 2` → `x := x + x`

13

13

Basic Block Level

- Common Subexpression elimination
- Constant Propagation
- Dead code elimination
- Plus many others such as copy propagation, value numbering, partial redundancy elimination, …

14

14

Simple example: a[i+1] = b[i+1]

- t1 = i+1
- t2 = b[t1]
- t3 = i + 1
- a[t3] = t2

- t1 = i + 1
- t2 = b[t1]
- t3 = i + 1
- a[t1] = t2

Common expression can be eliminated

15

15

Now, suppose i is a constant:

- i = 4
- t1 = i+1
- t2 = b[t1]
- a[t1] = t2

- i = 4
- t1 = 5
- t2 = b[t1]
- a[t1] = t2

- i = 4
- t1 = 5
- t2 = b[5]
- a[5] = t2

Final Code:
- i = 4
- t2 = b[5]
- a[5] = t2

16

16

Control Flow Graph - CFG

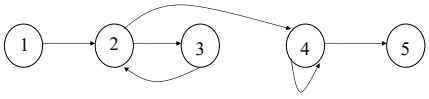
CFG = < V, E, Entry >, where

V = vertices or nodes, representing an instruction or basic block (group of statements).

E = (V x V) edges, potential flow of control

Entry is an element of V, the unique program entry

Two sets used in algorithms:
- Succ(v) = {x in V| exists e in E, e = v →x}
- Pred(v) = {x in V| exists e in E, e = x →v}



17

17

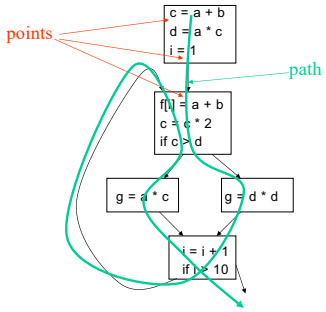
Definitions

- point - any location between adjacent statements and before and after a basic block.
- A path in a CFG from point p_1 to p_n is a sequence of points such that ∀ j, 1 <= j < n, either p_i is the point immediately preceding a statement and p_{i+1} is the point immediately following that statement in the same block, or p_i is the end of some block and p_{i+1} is the start of a successor block.

18

18

CFG



19

Optimizations on CFG

- Must take control flow into account
  - Common Sub-expression Elimination
  - Constant Propagation
  - Dead Code Elimination
  - Partial redundancy Elimination
  - …
- Applying one optimization may create opportunities for other optimizations.

20

Redundant Expressions

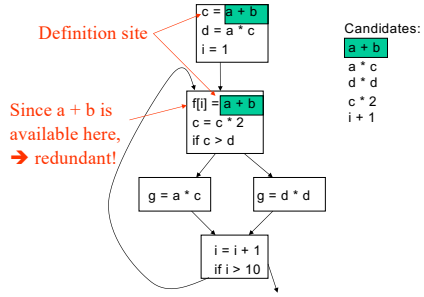
An expression **x op y** is redundant at a point p if it has already been computed at some point(s) and no intervening operations redefine **x** or **y**.

```
m = 2*y*z        t0 = 2*y         t0 = 2*y
                 m = t0*z         m = t0*z
n = 3*y*z        t1 = 3*y         t1 = 3*y
                 n = t1*z         n = t1*z
o = 2*y-z        t2 = 2*y
                 o = t2-z         o = t0-z
```

redundant

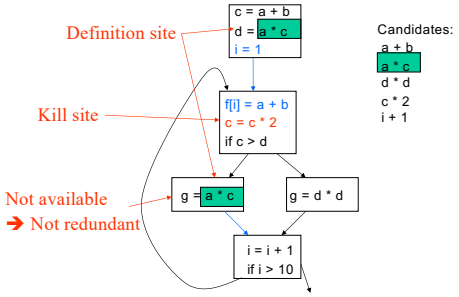
21

Redundant Expressions



22

Redundant Expressions



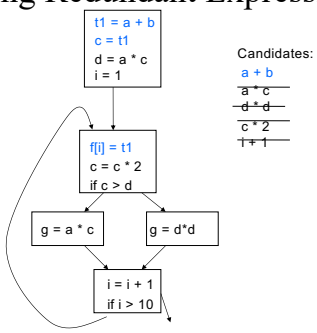
23

Redundant Expressions

- An expression *e* is defined at some point *p* in the CFG if its value is computed at *p*. (definition site)
- An expression *e* is killed at point *p* in the CFG if one or more of its operands is defined at *p*. (kill site)
- An expression is ***available*** at point *p* in a CFG if every path leading to *p* contains a prior definition of *e* and *e* is not killed between that definition and *p*.

24

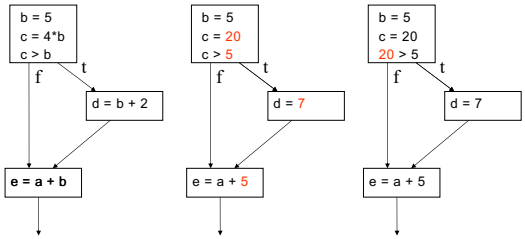
Removing Redundant Expressions



25

25

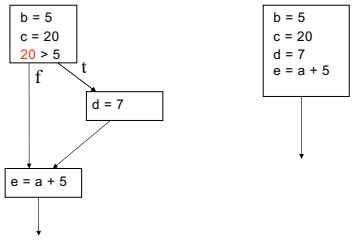
Constant Propagation



26

26

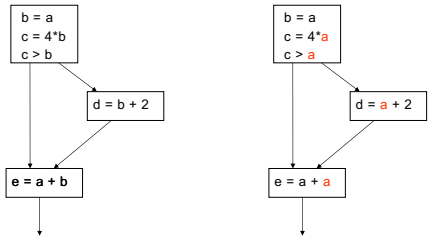
Constant Propagation



27

27

Copy Propagation



28

28

Simple Loop Optimizations: Code Motion

```
while (i <=  limit - 2)
```

```
L1:
    t1 = limit - 2
    if (i > t1) goto L2
    body of loop
    goto L1
L2:
```

```
t := limit - 2
   while (i <= t)
```

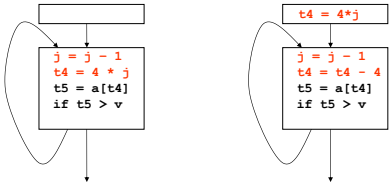
```
    t1 = limit - 2
L1:
    if (i > t1) goto L2
    body of loop
    goto L1
L2:
```

29

29

Simple Loop Optimizations: Strength Reduction

- Induction Variables control loop iterations



30

30

Simple Loop Optimizations

- Loop transformations are often used to expose other optimization opportunities:
  - Normalization
  - Loop Interchange
  - Loop Fusion
  - Loop Reversal
  - …

Global Data Flow Analysis

Collecting information about the way data is used in a program.

- Takes control flow into account
- HL control constructs
  - Simpler – syntax driven
  - Useful for data flow analysis of source code
- General control constructs – arbitrary branching

Information needed for optimizations such as: constant propagation, common sub-expressions, partial redundancy elimination …

Dataflow Analysis: Iterative Techniques

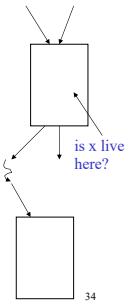
- First, compute local (block level) information.
- Iterate until no changes

```
while change do
    change = false
    for each basic block
          apply equations updating IN and OUT
          if either IN or OUT changes, set change
  to true
end
```


Live Variable Analysis

A variable **x** is *live* at a point p if there is some path from p where **x** is used before it is defined.

Want to determine for some variable x and point p whether the value of x *could* be used along some path starting at p.

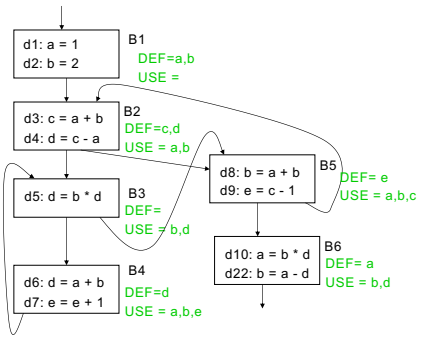


- Information flows backwards
- May – 'along some path starting at p'

Global Live Variable Analysis

Want to determine for some variable *x* and point *p* whether the value of *x* could be used along some path starting at *p*.

- DEF[B] - set of variables assigned values in B prior to any use of that variable
- USE[B] - set of variables used in B prior to any definition of that variable
- OUT[B] - variables live immediately after the block
  OUT[B] - $\cup$IN[S] for all S in succ(B)
- IN[B] - variables live immediately before the block
  IN[B] = USE[B] + (OUT[B] - DEF[B])



Global Live Variable Analysis

Want to determine for some variable x and point p whether the value of x could be used along some path starting at p.

- DEF[B] - set of variables assigned values in B prior to any use of that variable
- USE[B] - set of variables used in B prior to any definition of that variable
- OUT[B] - variables live immediately after the block
  OUT[B] - ∪ IN[S] for all S in succ(B)
- IN[B] - variables live immediately before the block
  IN[B] = USE[B] ∪ (OUT[B] - DEF[B])

37

37

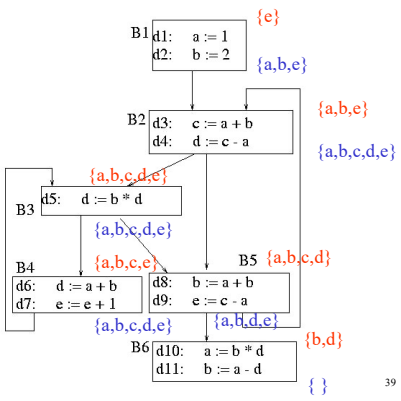
	IN	OUT	IN	OUT	IN	OUT
B1	∅	a,b	∅	a,b	e	a,b,e
B2	a,b	a,b,c,d	a,b,e	a,b,c,d ,e	a,b,e	a,b,c,d,e
B3	a,b,c,d e	a,b,c,e	a,b,c,d,e	a,b,c,d,e	a,b,c,d,e	a,b,c,d,e
B4	a,b,c,e	a,b,c,d,e	a,b,c,e	a,b,c,d,e	a,b,c,e	a,b,c,d,e
B5	a,b,c,d	a,b,d	a,b,c,d	a,b,d,e	a,b,c,d	a,b,d,e
B6	b,d	∅	b,d	∅	b,d	∅

OUT[B] = ∪ IN[S] for all S in succ(B)
IN[B] = USE[B] + (OUT[B] - DEF[B])

Block	DEF	USE
B1	{a,b}	{ }
B2	{c,d}	{a,b}
B3	{ }	{b,d}
B4	{d}	{a,b,e}
B5	{e}	{a,b,c}
B6	{a}	{b,d}

38

38



39

39

Local vs. Global analysis

1. In a local analysis, each statement has exactly one predecessor. In a global analysis, each statement may have multiple predecessors.
   - ➔ A global analysis must have some means of combining information from all predecessors of a basic block
2. In a local analysis, there is only one possible path through a basic block. In a global analysis, there may be many paths through a CFG.
   - ➔May need to recompute values multiple times as more information becomes available. Need to be careful when doing this not to loop infinitely!

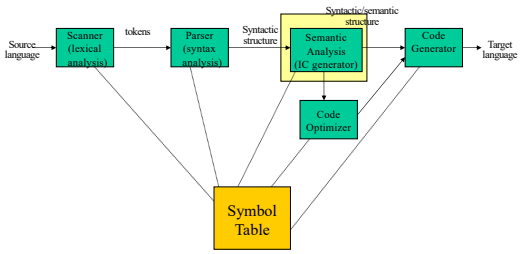
40

Local vs. Global analysis

3. In a local analysis, there is always a well-defined "first" statement to begin processing. In a global analysis with loops, every basic block might depend on every other basic block.
   - ➔To fix this, we need to assign initial values to all of the blocks in the CFG.

41

Compiler



42